

一、 树的基本概念

1. 树的定义

树是 n 个节点的有限集合。

- 当 $n = 0$ 时，称为**空树**。
- 当 $n > 0$ 时，满足：
 - 有且仅有一个特定的节点称为**根节点** (Root)，它没有前驱。
 - 其余节点可分为 m ($m \geq 0$) 个互不相交的有限集合 $T_1, T_2, \dots, T_m$ ，其中每一个集合本身又是一棵树，并且称为根节点的**子树** (Subtree)。

2. 树的特性

- 树是一种**递归定义**的数据结构。
- 树中的节点有且仅有一个前驱（双亲），但可以有零个或多个后继（孩子）。
- 子树之间**互不相交**（即没有公共节点）。
- 树中任意两个节点之间有且仅有一条路径。

二、 树的基本术语

术语	定义
节点	树中的每个元素称为节点。
双亲（父节点）	一个节点的直接前驱称为该节点的双亲。
孩子（子节点）	一个节点的直接后继称为该节点的孩子。
兄弟	具有相同双亲的节点互称为兄弟。
堂兄弟	双亲在同一层的节点互称为堂兄弟。
祖先	从根节点到该节点路径上的所有节点（包括该节点自身）称为该节点的祖先。
子孙	以该节点为根的子树中的所有节点（包括该节点自身）称为该节点的子孙。
节点的度	节点拥有的子树个数（即孩子的个数）。
树的度	树内各节点度的最大值。
叶子节点（终端节点）	度为 0 的节点。
分支节点（非终端节点）	度大于 0 的节点。
内部节点	除根节点以外的分支节点（根节点可以是分支节点）。
节点的层次	根节点为第 1 层，根节点的孩子为第 2 层，依此类推。
树的深度（高度）	树中节点的最大层次。注意：有的教材从 0 开始计数，但考研通常从 1 开始。

术语	定义
路径	从节点 n_1 到 n_k 经过的节点序列 $n_1, n_2, \dots, n_k$ 。
路径长度	路径上经过的 边的条数 。
有序树 vs 无序树	若树中节点的各子树从左到右有次序（不可互换），则为有序树；否则为无序树。
森林	m ($m \geq 0$) 棵互不相交的树的集合。对树中每个节点而言，其子树的集合就是森林。

三、1234 树的重要性质与考点

性质 1

树中的节点数 = 所有节点的度数之和 + 1
 （因为根节点没有双亲，而其他节点都有一个双亲，总度数等于除了根以外所有节点的个数）

$$\text{总结点数} = \text{总度数} + 1 = \text{总边数} + 1$$

性质 2

度为 m 的树中，第 i 层上最多有 $m^{(i-1)}$ 个节点 ($i \geq 1$)。

性质 3

高度为 h 的 m 叉树最多有 $(m^h - 1)/(m - 1)$ 个节点（满 m 叉树的情况）。

性质 4

具有 n 个节点的 m 叉树的最小高度为 $\lceil \log_m (n(m-1) + 1) \rceil$ 。

⚠ 高频易错点：度为 m 的树 vs m 叉树

对比项	度为 m 的树	m 叉树
定义	树中 最大度数 为 m （即至少有一个节点的度等于 m ）	每个节点 最多 有 m 个孩子（可以所有节点度都小于 m ）
空树	一定是 非空树 （树至少有一个节点）	可以是空树
节点数下限	高度为 h 时，至少有 $h + m - 1$ 个节点	高度为 h 时，至少有 h 个节点（每层一个）
典型例子	度为 3 的树，必须有一个节点有 3 个孩子	三叉树，可以所有节点只有 0~2 个孩子

典型例题

例题 1：求含有 n 个节点的三叉树的**最小高度**。

解题思路：为使高度最小，应尽量让每一层的节点数达到最大（即排满）。设高度为 h ，则节点数满足 $n \leq (3^h - 1)/(3 - 1) = (3^h - 1)/2$ ，解得 $h \geq \lceil \log_3(2n + 1) \rceil$ 。

例题 2：已知一棵度为 4 的树中，度为 0, 1, 2, 3 的节点数分别为 14, 4, 3, 2，求总结点数 n 和度为 4 的节点个数 n_4 。

解题思路：设总节点数为 n ，总度数为 D 。则

$$n = n_0 + n_1 + n_2 + n_3 + n_4 = 14 + 4 + 3 + 2 + n_4 = 23 + n_4$$

又

$$D = 0 \times n_0 + 1 \times n_1 + 2 \times n_2 + 3 \times n_3 + 4 \times n_4 = 0 + 4 + 6 + 6 + 4n_4 = 16 + 4n_4$$

由性质 1（节点数 = 总度数 + 1），得

$$23 + n_4 = (16 + 4n_4) + 1 \Rightarrow n_4 = 2$$

进而 $n = 25$ 。

例题 3：一棵有 2023 个叶子的 3 叉树，最少有多少个节点？。

解题思路：根据经验可知，想要结点数量最少，树应该尽量是满叉树。我们把结点分成两类：叶子结点和内部结点，其中叶子结点称作 L ，内部节点称作 I ，树的叉数为 M ，则 $I = \lceil (L - 1) / (M - 1) \rceil$ 。则总结点数 = $I + L$ 。则：

$$I = (2023 - 1) / (3 - 1) = 1011$$

$$\text{总结点数} = I + L = 2023 + 1011 = 3034$$

四、🌲 二叉树 (Binary Tree)

1. 二叉树的定义

二叉树是 n ($n \geq 0$) 个节点的有限集合，它要么是空集 ($n = 0$)，要么由一个根节点和两棵互不相交的、分别称为**左子树**和**右子树**的二叉树组成。

2. 二叉树的特点

- 每个节点至多有两棵子树，即不存在度大于 2 的节点。
- 二叉树的子树仍然是二叉树。

3. 二叉树的性质（重要考点）

性质	内容
性质 1	在二叉树的第 i 层上至多有 $2^{(i-1)}$ 个节点 ($i \geq 1$)。
性质 2	深度为 h 的二叉树至多有 $2^h - 1$ 个节点 ($h \geq 1$)。
性质 3	对任意一棵二叉树，若叶子节点数为 n_0 ，度为 2 的节点数为 n_2 ，则 $n_0 = n_2 + 1$ 。 (推导：总节点数 $n = n_0 + n_1 + n_2$ ，总度数 $n-1 = n_1 + 2n_2$ ，两式相减得 $n_0 = n_2 + 1$)
性质 4	具有 n 个节点的完全二叉树深度为 $\lfloor \log_2 n \rfloor + 1$ 。
性质 5（顺序存储编号）	对完全二叉树按从上到下、从左到右编号（从 1 开始），则： - 节点 i 的双亲为 $\lfloor i/2 \rfloor$ ($i > 1$) - 左孩子为 $2i$ (若 $2i \leq n$) - 右孩子为 $2i+1$ (若 $2i+1 \leq n$)

4. 二叉树的存储结构

(1) 顺序存储结构

- 用一组地址连续的存储单元依次存放二叉树中的节点。
- 对于完全二叉树和满二叉树，可以紧凑存储且不浪费空间。
- 对于一般二叉树，需要添加“空节点”占位，可能导致大量空间浪费。

(2) 链式存储结构（最常用）

- **二叉链表**：每个节点包含三个域——数据域（data）、左孩子指针（lchild）、右孩子指针（rchild）。
- 在 n 个节点的二叉链表中，有 $n+1$ 个空指针（因为总指针域为 $2n$ ，其中非空指针为 $n-1$ 条边）。
- **三叉链表**：在二叉链表基础上增加一个指向双亲的指针域（parent），方便向上查找。

五、🎯 几种特殊的二叉树

1. 满二叉树 (Full Binary Tree)

- **定义**：一棵深度为 h ，且含有 $2^h - 1$ 个节点的二叉树。
- **特点**：
 - 每一层上的节点数都达到最大值。
 - 叶子节点全部在最底层。
 - 节点编号可以自上而下、自左至右连续编号。

2. 完全二叉树 (Complete Binary Tree)

- **定义**：深度为 h 、有 n 个节点的二叉树，当且仅当其每一个节点都与深度为 h 的满二叉树中编号从 1 到 n 的节点——对应。
- **特点**：
 - 叶子节点只可能出现在最后两层。
 - 最后一层的叶子节点都**靠左对齐**（即若某节点有右孩子，则必有左孩子）。
 - 对于任意节点，若其右子树深度为 l ，则其左子树深度要么为 l ，要么为 $l+1$ 。
 - 适合用数组顺序存储。

3. 二叉排序树 (Binary Sort Tree, BST), 也称二叉查找树

- **定义**：一棵空树，或者满足：
 - 若左子树非空，则左子树上所有节点的值均小于根节点的值。
 - 若右子树非空，则右子树上所有节点的值均大于根节点的值。
 - 左、右子树本身也分别是二叉排序树。
- **作用**：实现高效的动态查找、插入和删除。中序遍历二叉排序树可得到一个递增的有序序列。

六、 哈夫曼树 (Huffman Tree)

1. 哈夫曼树的定义

- **带权路径长度 (Weighted Path Length, WPL)**：树中所有叶子节点的权值乘以其到根节点的路径长度（边的条数）之和。
- **哈夫曼树**：在给定 n 个权值作为叶子节点构造的二叉树中，带权路径长度最小的二叉树称为哈夫曼树，也称**最优二叉树**。

2. 哈夫曼树的构造算法（贪心思想）

1. 将给定的 n 个权值视为 n 棵只有根节点的二叉树，构成森林。
2. 在森林中选取两棵根节点权值**最小**的树作为左右子树，构造一棵新的二叉树，新树根节点的权值为左右子树根节点权值之和。
3. 从森林中删除这两棵树，并将新树加入森林。

4. 重复步骤 2 和 3，直到森林中只剩一棵树，该树即为哈夫曼树。

注意：

- 构造时左右子树顺序可以交换（不影响 WPL，但影响编码）。
- 哈夫曼树不唯一，但带权路径长度最小且唯一。
- 哈夫曼树中没有度为 1 的节点，即 $n_0 = n_2 + 1$ ，且总结点数为 $2n - 1$ 。

3. 哈夫曼编码

- **前缀编码**：任意一个编码都不是其他编码的前缀，这样解码时不会产生歧义。
- **哈夫曼编码**：利用哈夫曼树进行编码。约定左分支为 0，右分支为 1（或相反），从根到叶子节点的路径上 0/1 序列即为该叶子节点对应字符的编码。
- **特点**：哈夫曼编码是**最优前缀编码**，能使报文总长度最短（即 WPL 最小）。

4. 经典例题

例：给定权值 {5, 29, 7, 8, 14, 23, 3, 11}，构造哈夫曼树并计算 WPL。

解题步骤：反复取最小两个权值合并，最终 $WPL = (3+5) \times 4 + (7+8+11) \times 3 + (14+23+29) \times 2 = \dots$ （具体计算略）。

5. 哈夫曼树的性质总结

性质	说明
节点度数为 0 或 2	没有度为 1 的节点
总结点数	$2n - 1$ (n 为叶子节点数)
不唯一性	左右子树交换、权值相等时不同选择可产生不同树
WPL 最小性	是有可能二叉树中最小的
应用	数据压缩（哈夫曼编码）、最优判定树等

七、📌 总结与核心考点归纳

- 树的基本术语**：度、叶子、双亲、孩子、深度、森林等必须熟练掌握。
- 树的性质**：节点数 = 总度数 + 1；度为 m 的树与 m 叉树的区别是选择题高频考点。
- 二叉树性质**：尤其是 $n_0 = n_2 + 1$ 和完全二叉树编号公式，经常出现在大题中。
- 特殊二叉树**：满二叉树、完全二叉树、二叉排序树、平衡二叉树的定义和特性要清晰。
- 哈夫曼树**：构造过程、WPL 计算、哈夫曼编码的生成与应用，是数据压缩部分的重点。